Proyecto I – Aprendizaje por Reflejo

CC3085 – Inteligencia Artificial

Universidad del Valle de Guatemala

Integrantes:

- Felipe Aguilar
- Ernesto Ascencio

Objetivo: Predecir el precio de venta (`SalePrice`) de casas utilizando al menos 2 algoritmos de regresión supervisada. Se realizará un análisis exploratorio de datos (EDA), preprocesamiento, entrenamiento, comparación de modelos y persistencia del mejor modelo con `pickle` .

---

1. Importación de Librerías

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import warnings

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Modelos
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.tree import DecisionTreeRegressor

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

print('Librerías importadas correctamente.')
```

```
Librerías importadas correctamente.
```

2. Carga de Datos

In [2]:
```
features = pd.read_csv('training_features.csv')
target = pd.read_csv('training_target.csv')

# Unir features y target en un solo DataFrame
df = pd.concat([features, target], axis=1)
```

```
print(f'Dimensiones features: {features.shape}')
print(f'Dimensiones target:   {target.shape}')
print(f'Dimensiones unidas:   {df.shape}')
print(f'\nPrimeras 5 filas:')
df.head()
```

```
Dimensiones features: (2344, 79)
Dimensiones target:   (2344, 1)
Dimensiones unidas:   (2344, 80)

Primeras 5 filas:
```

Out[2]:

	MS SubClass	MS Zoning	Lot Frontage	Lot Area	Street	Alley	Lot Shape	Land Contour	Utilities	Lot Config	...	Pool Area	Pool QC	Fence	Misc Feature	Misc Val	Mo Sold	Yr Sold	Sale Type	Sale Condition	SalePrice
0	20	RL	80.0	10400	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	MnPrv	NaN	0	6	2009	WD	Family	152000
1	60	RL	NaN	28698	Pave	NaN	IR2	Low	AllPub	CulDSac	...	0	NaN	NaN	NaN	0	6	2009	WD	Abnorml	185000
2	90	RL	70.0	9842	Pave	NaN	Reg	Lvl	AllPub	FR2	...	0	NaN	NaN	NaN	0	3	2007	WD	Normal	101800
3	90	RL	60.0	7200	Pave	NaN	Reg	Lvl	AllPub	Inside	...	0	NaN	NaN	NaN	0	6	2009	WD	Normal	90000
4	190	RM	63.0	7627	Pave	NaN	Reg	Lvl	AllPub	Corner	...	0	NaN	NaN	NaN	0	10	2009	WD	Normal	94550

5 rows × 80 columns

---

3. Análisis Exploratorio de Datos (EDA)

3.1 Información General del Dataset

In [3]:
```
print('='*60)
print('INFORMACIÓN GENERAL DEL DATASET')
print('='*60)
print(f'Filas: {df.shape[0]}')
print(f'Columnas: {df.shape[1]}')
print(f'\nColumnas numéricas:   {df.select_dtypes(include=[np.number]).shape[1]}')
print(f'Columnas categóricas: {df.select_dtypes(include=["object"]).shape[1]}')
print(f'\nTipos de datos:')
print(df.dtypes.value_counts())
```

```
============================================================
INFORMACIÓN GENERAL DEL DATASET
============================================================
Filas: 2344
Columnas: 80

Columnas numéricas:   37
Columnas categóricas: 43

Tipos de datos:
str        43
int64      26
float64    11
Name: count, dtype: int64
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2344 entries, 0 to 2343
Data columns (total 80 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   MS SubClass      2344 non-null   int64
 1   MS Zoning        2344 non-null   str
 2   Lot Frontage     1951 non-null   float64
 3   Lot Area         2344 non-null   int64
 4   Street           2344 non-null   str
 5   Alley            162 non-null    str
 6   Lot Shape        2344 non-null   str
 7   Land Contour     2344 non-null   str
 8   Utilities        2344 non-null   str
 9   Lot Config       2344 non-null   str
 10  Land Slope       2344 non-null   str
 11  Neighborhood     2344 non-null   str
 12  Condition 1      2344 non-null   str
 13  Condition 2      2344 non-null   str
 14  Bldg Type        2344 non-null   str
 15  House Style      2344 non-null   str
 16  Overall Qual     2344 non-null   int64
 17  Overall Cond     2344 non-null   int64
 18  Year Built       2344 non-null   int64
 19  Year Remod/Add   2344 non-null   int64
 20  Roof Style       2344 non-null   str
 21  Roof Matl        2344 non-null   str
 22  Exterior 1st     2344 non-null   str
 23  Exterior 2nd     2344 non-null   str
 24  Mas Vnr Type     918 non-null    str
 25  Mas Vnr Area     2325 non-null   float64
 26  Exter Qual       2344 non-null   str
 27  Exter Cond       2344 non-null   str
 28  Foundation       2344 non-null   str
 29  Bsmt Qual        2283 non-null   str
 30  Bsmt Cond        2283 non-null   str
 31  Bsmt Exposure    2281 non-null   str
 32  BsmtFin Type 1   2283 non-null   str
 33  BsmtFin SF 1     2343 non-null   float64
 34  BsmtFin Type 2   2282 non-null   str
 35  BsmtFin SF 2     2343 non-null   float64
 36  Bsmt Unf SF      2343 non-null   float64
 37  Total Bsmt SF    2343 non-null   float64
 38  Heating          2344 non-null   str
 39  Heating QC       2344 non-null   str
 40  Central Air      2344 non-null   str
 41  Electrical       2344 non-null   str
 42  1st Flr SF       2344 non-null   int64
 43  2nd Flr SF       2344 non-null   int64
 44  Low Qual Fin SF  2344 non-null   int64
 45  Gr Liv Area      2344 non-null   int64
 46  Bsmt Full Bath   2343 non-null   float64
 47  Bsmt Half Bath   2343 non-null   float64
 48  Full Bath        2344 non-null   int64
 49  Half Bath        2344 non-null   int64
 50  Bedroom AbvGr    2344 non-null   int64
 51  Kitchen AbvGr    2344 non-null   int64
 52  Kitchen Qual     2344 non-null   str
 53  TotRms AbvGrd    2344 non-null   int64
```

```
 54  Functional       2344 non-null   str
 55  Fireplaces       2344 non-null   int64
 56  Fireplace Qu     1200 non-null   str
 57  Garage Type      2224 non-null   str
 58  Garage Yr Blt    2222 non-null   float64
 59  Garage Finish    2222 non-null   str
 60  Garage Cars      2343 non-null   float64
 61  Garage Area      2343 non-null   float64
 62  Garage Qual      2222 non-null   str
 63  Garage Cond      2222 non-null   str
 64  Paved Drive      2344 non-null   str
 65  Wood Deck SF     2344 non-null   int64
 66  Open Porch SF    2344 non-null   int64
 67  Enclosed Porch   2344 non-null   int64
 68  3Ssn Porch       2344 non-null   int64
 69  Screen Porch     2344 non-null   int64
 70  Pool Area        2344 non-null   int64
 71  Pool QC          12 non-null     str
 72  Fence            470 non-null    str
 73  Misc Feature     94 non-null     str
 74  Misc Val         2344 non-null   int64
 75  Mo Sold          2344 non-null   int64
 76  Yr Sold          2344 non-null   int64
 77  Sale Type        2344 non-null   str
 78  Sale Condition   2344 non-null   str
 79  SalePrice        2344 non-null   int64
dtypes: float64(11), int64(26), str(43)
memory usage: 1.4 MB
```

In [5]:
```
df.describe().T
```

Out[5]:

	count	mean	std	min	25%	50%	75%	max
MS SubClass	2344.0	57.308020	42.802555	20.0	20.00	50.0	70.00	190.0
Lot Frontage	1951.0	69.207586	23.418018	21.0	58.00	68.0	80.00	313.0
Lot Area	2344.0	10127.857509	8050.908132	1300.0	7466.50	9356.5	11484.25	215245.0
Overall Qual	2344.0	6.064420	1.388520	1.0	5.00	6.0	7.00	10.0
Overall Cond	2344.0	5.581911	1.105659	1.0	5.00	5.0	6.00	9.0
Year Built	2344.0	1970.506826	30.341434	1872.0	1953.00	1972.0	2000.00	2010.0
Year Remod/Add	2344.0	1983.923635	20.786287	1950.0	1965.00	1992.0	2003.00	2010.0
Mas Vnr Area	2325.0	98.539785	172.239113	0.0	0.00	0.0	160.00	1290.0
BsmtFin SF 1	2343.0	442.531370	452.223224	0.0	0.00	375.0	734.00	5644.0
BsmtFin SF 2	2343.0	50.097311	170.396999	0.0	0.00	0.0	0.00	1526.0
Bsmt Unf SF	2343.0	554.393513	433.788333	0.0	218.00	462.0	795.50	2336.0
Total Bsmt SF	2343.0	1047.022194	436.567117	0.0	784.00	988.0	1288.00	6110.0
1st Flr SF	2344.0	1154.814420	385.114269	334.0	879.75	1082.0	1378.00	5095.0
2nd Flr SF	2344.0	333.688140	427.141191	0.0	0.00	0.0	701.00	2065.0
Low Qual Fin SF	2344.0	4.357082	44.323993	0.0	0.00	0.0	0.00	1064.0
Gr Liv Area	2344.0	1492.859642	504.619676	334.0	1121.75	1436.5	1734.00	5642.0
Bsmt Full Bath	2343.0	0.431498	0.525505	0.0	0.00	0.0	1.00	3.0
Bsmt Half Bath	2343.0	0.059326	0.241644	0.0	0.00	0.0	0.00	2.0
Full Bath	2344.0	1.557167	0.552162	0.0	1.00	2.0	2.00	4.0
Half Bath	2344.0	0.370734	0.499608	0.0	0.00	0.0	1.00	2.0
Bedroom AbvGr	2344.0	2.845137	0.820352	0.0	2.00	3.0	3.00	8.0
Kitchen AbvGr	2344.0	1.043089	0.211339	0.0	1.00	1.0	1.00	3.0
TotRms AbvGrd	2344.0	6.409130	1.563651	2.0	5.00	6.0	7.00	15.0
Fireplaces	2344.0	0.598976	0.652987	0.0	0.00	1.0	1.00	4.0
Garage Yr Blt	2222.0	1977.398290	25.679110	1895.0	1960.00	1978.0	2001.00	2207.0
Garage Cars	2343.0	1.746906	0.746445	0.0	1.00	2.0	2.00	4.0
Garage Area	2343.0	469.078959	212.432786	0.0	319.00	476.0	576.00	1488.0
Wood Deck SF	2344.0	94.113055	124.851870	0.0	0.00	0.0	168.00	870.0
Open Porch SF	2344.0	47.341724	68.037465	0.0	0.00	26.0	70.00	742.0
Enclosed Porch	2344.0	24.731655	67.030943	0.0	0.00	0.0	0.00	1012.0
3Ssn Porch	2344.0	2.422782	24.524362	0.0	0.00	0.0	0.00	508.0
Screen Porch	2344.0	16.010239	55.820882	0.0	0.00	0.0	0.00	576.0
Pool Area	2344.0	2.582765	38.324145	0.0	0.00	0.0	0.00	800.0
Misc Val	2344.0	58.055034	623.375121	0.0	0.00	0.0	0.00	17000.0

	count	mean	std	min	25%	50%	75%	max
Mo Sold	2344.0	6.184727	2.708408	1.0	4.00	6.0	8.00	12.0
Yr Sold	2344.0	2007.794795	1.315117	2006.0	2007.00	2008.0	2009.00	2010.0
SalePrice	2344.0	178582.207765	77125.072713	12789.0	129000.00	160000.0	210000.00	755000.0

3.2 Análisis de Valores Faltantes

In [6]:
```
# Valores faltantes por columna
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
missing_df = pd.DataFrame({'Faltantes': missing, '% Faltantes': missing_pct})
missing_df = missing_df[missing_df['Faltantes'] > 0].sort_values('% Faltantes', ascending=False)

print(f'Columnas con valores faltantes: {len(missing_df)} de {df.shape[1]}')
print(f'Total de valores faltantes: {df.isnull().sum().sum()}')
print()
missing_df
```

```
Columnas con valores faltantes: 26 de 80
Total de valores faltantes: 12544
```

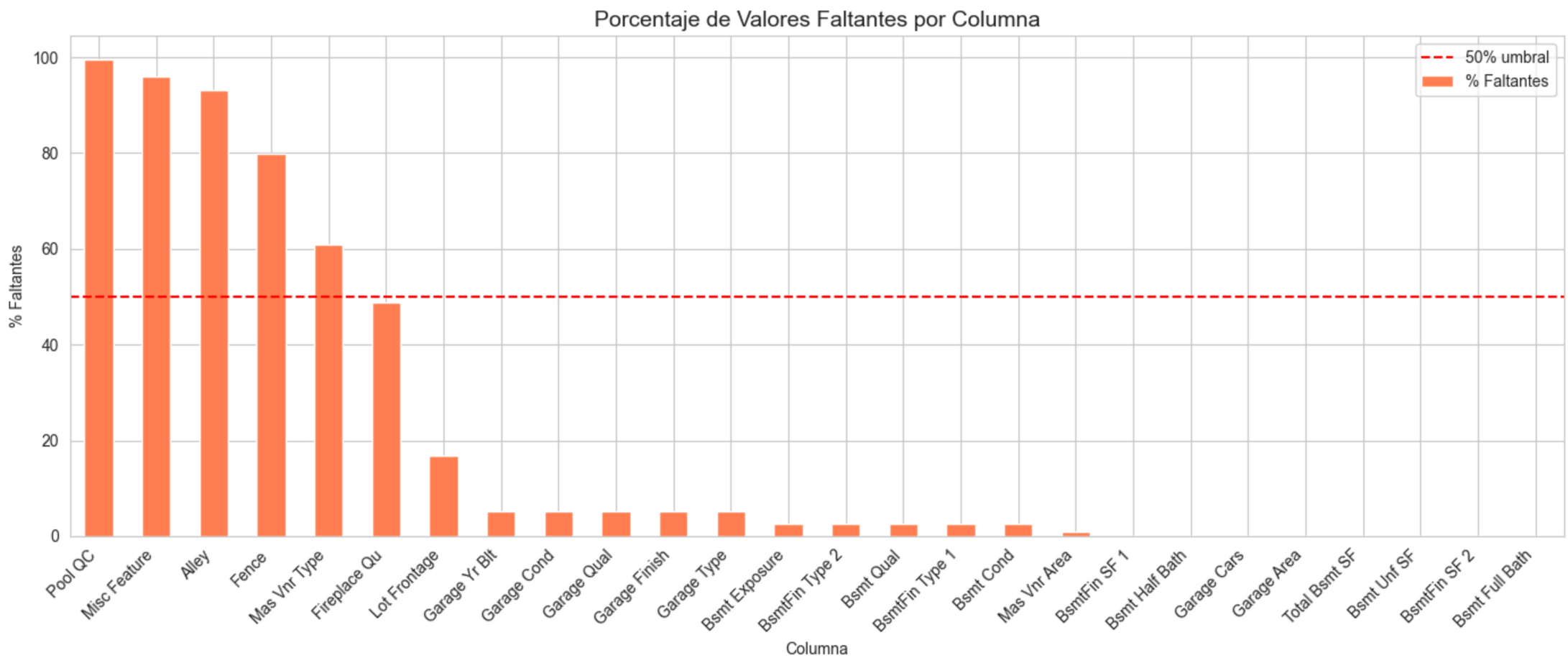
Out[6]:

	Faltantes	% Faltantes
Pool QC	2332	99.488055
Misc Feature	2250	95.989761
Alley	2182	93.088737
Fence	1874	79.948805
Mas Vnr Type	1426	60.836177
Fireplace Qu	1144	48.805461
Lot Frontage	393	16.766212
Garage Yr Blt	122	5.204778
Garage Cond	122	5.204778
Garage Qual	122	5.204778
Garage Finish	122	5.204778
Garage Type	120	5.119454
Bsmt Exposure	63	2.687713
BsmtFin Type 2	62	2.645051
Bsmt Qual	61	2.602389
BsmtFin Type 1	61	2.602389
Bsmt Cond	61	2.602389
Mas Vnr Area	19	0.810580
BsmtFin SF 1	1	0.042662
Bsmt Half Bath	1	0.042662
Garage Cars	1	0.042662
Garage Area	1	0.042662
Total Bsmt SF	1	0.042662
Bsmt Unf SF	1	0.042662
BsmtFin SF 2	1	0.042662
Bsmt Full Bath	1	0.042662

In [7]:

```
# Visualización de valores faltantes
if len(missing_df) > 0:
    fig, ax = plt.subplots(figsize=(14, 6))
    missing_df['% Faltantes'].plot(kind='bar', color='coral', ax=ax)
    ax.set_title('Porcentaje de Valores Faltantes por Columna', fontsize=14)
    ax.set_ylabel('% Faltantes')
    ax.set_xlabel('Columna')
    ax.axhline(y=50, color='red', linestyle='--', label='50% umbral')
    ax.legend()
    plt.xticks(rotation=45, ha='right')
```

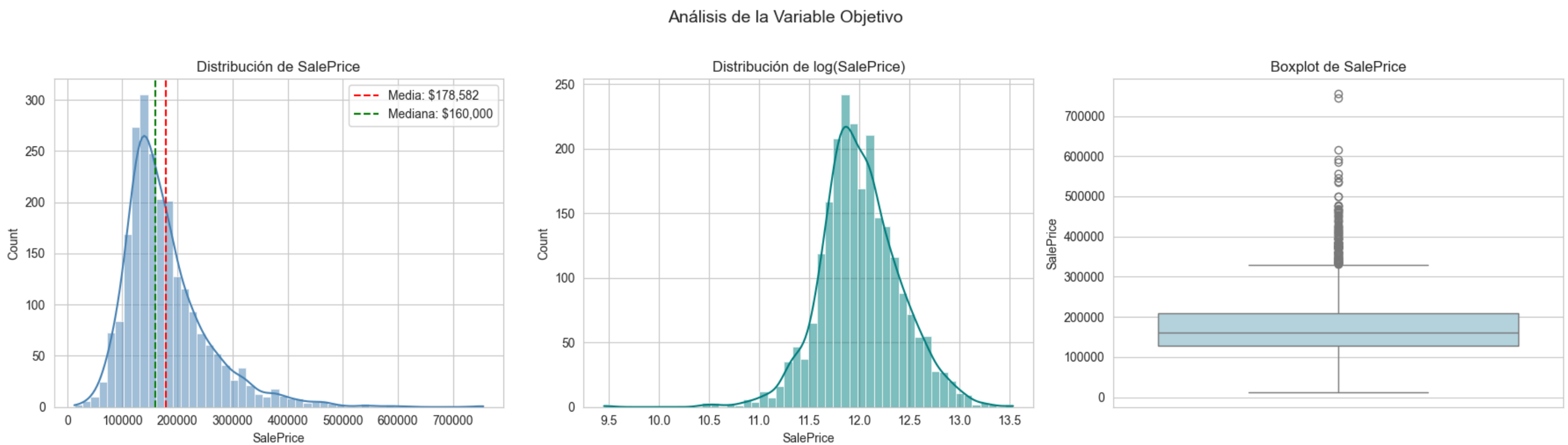
```
plt.tight_layout()
plt.show()
```



3.3 Análisis de la Variable Objetivo (SalePrice)

In [8]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Distribución
sns.histplot(df['SalePrice'], kde=True, bins=50, ax=axes[0], color='steelblue')
axes[0].set_title('Distribución de SalePrice')
axes[0].axvline(df['SalePrice'].mean(), color='red', linestyle='--', label=f'Media: ${df["SalePrice"].mean():,.0f}')
axes[0].axvline(df['SalePrice'].median(), color='green', linestyle='--', label=f'Mediana: ${df["SalePrice"].median():,.0f}')
axes[0].legend()

# Distribución log
sns.histplot(np.log1p(df['SalePrice']), kde=True, bins=50, ax=axes[1], color='teal')
axes[1].set_title('Distribución de log(SalePrice)')

# Boxplot
sns.boxplot(y=df['SalePrice'], ax=axes[2], color='lightblue')
axes[2].set_title('Boxplot de SalePrice')

plt.suptitle('Análisis de la Variable Objetivo', fontsize=14, y=1.02)
```

```
plt.tight_layout()
plt.show()

print(f'Media:     ${df["SalePrice"].mean():,.2f}')
print(f'Mediana:   ${df["SalePrice"].median():,.2f}')
print(f'Desv. Std: ${df["SalePrice"].std():,.2f}')
print(f'Skewness:  {df["SalePrice"].skew():.4f}')
print(f'Kurtosis:  {df["SalePrice"].kurtosis():.4f}')
```




```
Media:     $178,582.21
Mediana:   $160,000.00
Desv. Std: $77,125.07
Skewness:  1.7506
Kurtosis:  5.4745
```

Observación: La distribución de `SalePrice` presenta sesgo positivo (right-skewed). La transformación logarítmica produce una distribución más cercana a la normal, lo cual puede beneficiar ciertos modelos de regresión.

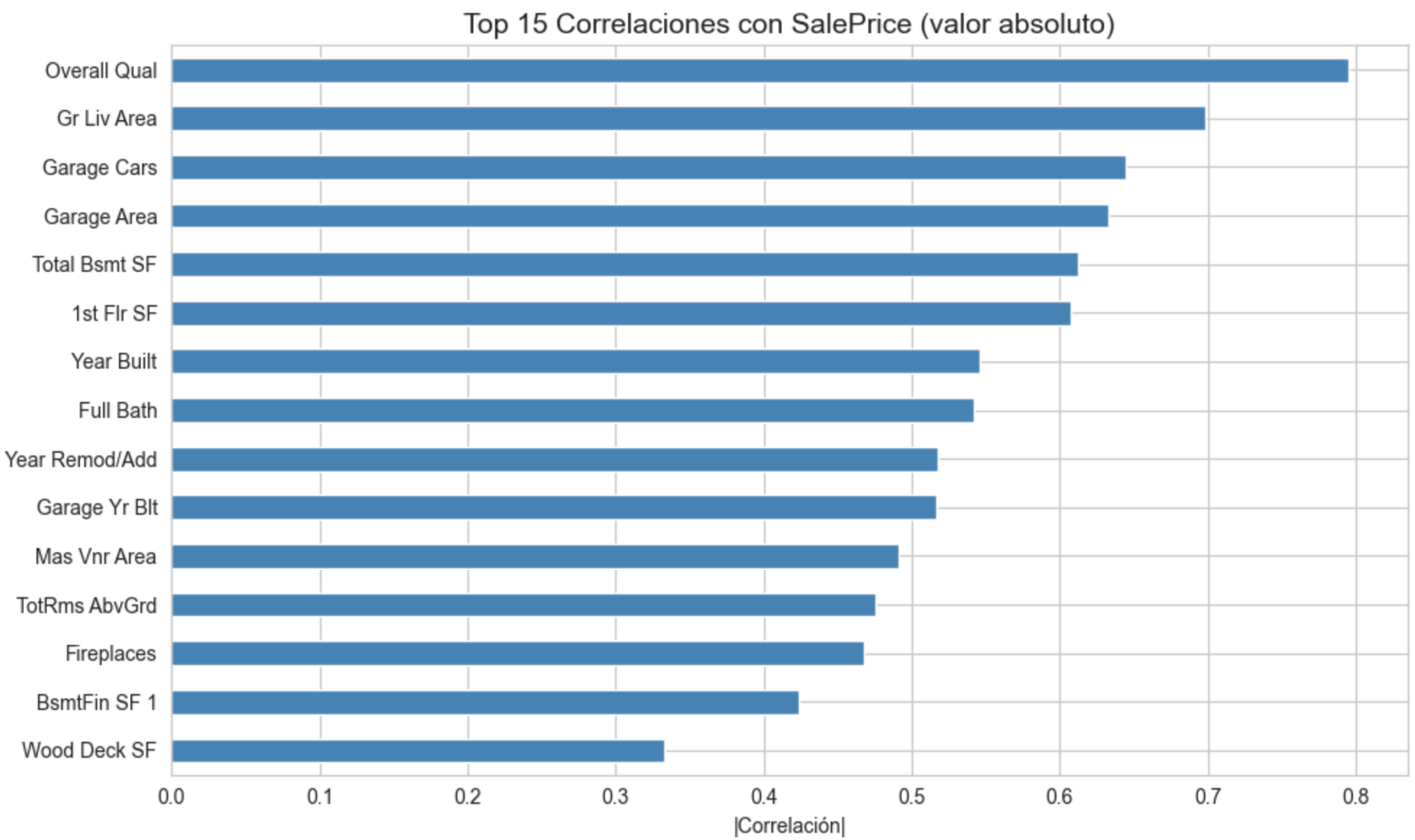
3.4 Variables Numéricas – Distribuciones y Outliers

In [9]:
```
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
num_cols_no_target = [c for c in num_cols if c != 'SalePrice']

print(f'Variables numéricas (sin target): {len(num_cols_no_target)}')
print(num_cols_no_target)
```

```
Variables numéricas (sin target): 36
['MS SubClass', 'Lot Frontage', 'Lot Area', 'Overall Qual', 'Overall Cond', 'Year Built', 'Year Remod/Add', 'Mas Vnr Area', 'BsmtFin SF 1', 'BsmtFin SF 2', 'Bsmt Unf SF', 'Total Bsmt SF', '1st Flr SF', '2nd Flr SF', 'Low Qual Fin SF', 'Gr Liv Area', 'Bsmt Full Bath', 'Bsmt Half Bath', 'Full Bath', 'Half Bath', 'Bedroom AbvGr', 'Kitchen AbvGr', 'TotRms AbvGrd', 'Fireplaces', 'Garage Yr Blt', 'Garage Cars', 'Garage Area', 'Wood Deck SF', 'Open Porch SF', 'Enclosed Porch', '3Ssn Porch', 'Screen Porch', 'Pool Area', 'Misc Val', 'Mo Sold', 'Yr Sold']
```

In [10]:
```
# Top 15 correlaciones con SalePrice
corr_with_target = df[num_cols].corr()['SalePrice'].drop('SalePrice').abs().sort_values(ascending=False)
top_corr = corr_with_target.head(15)
```

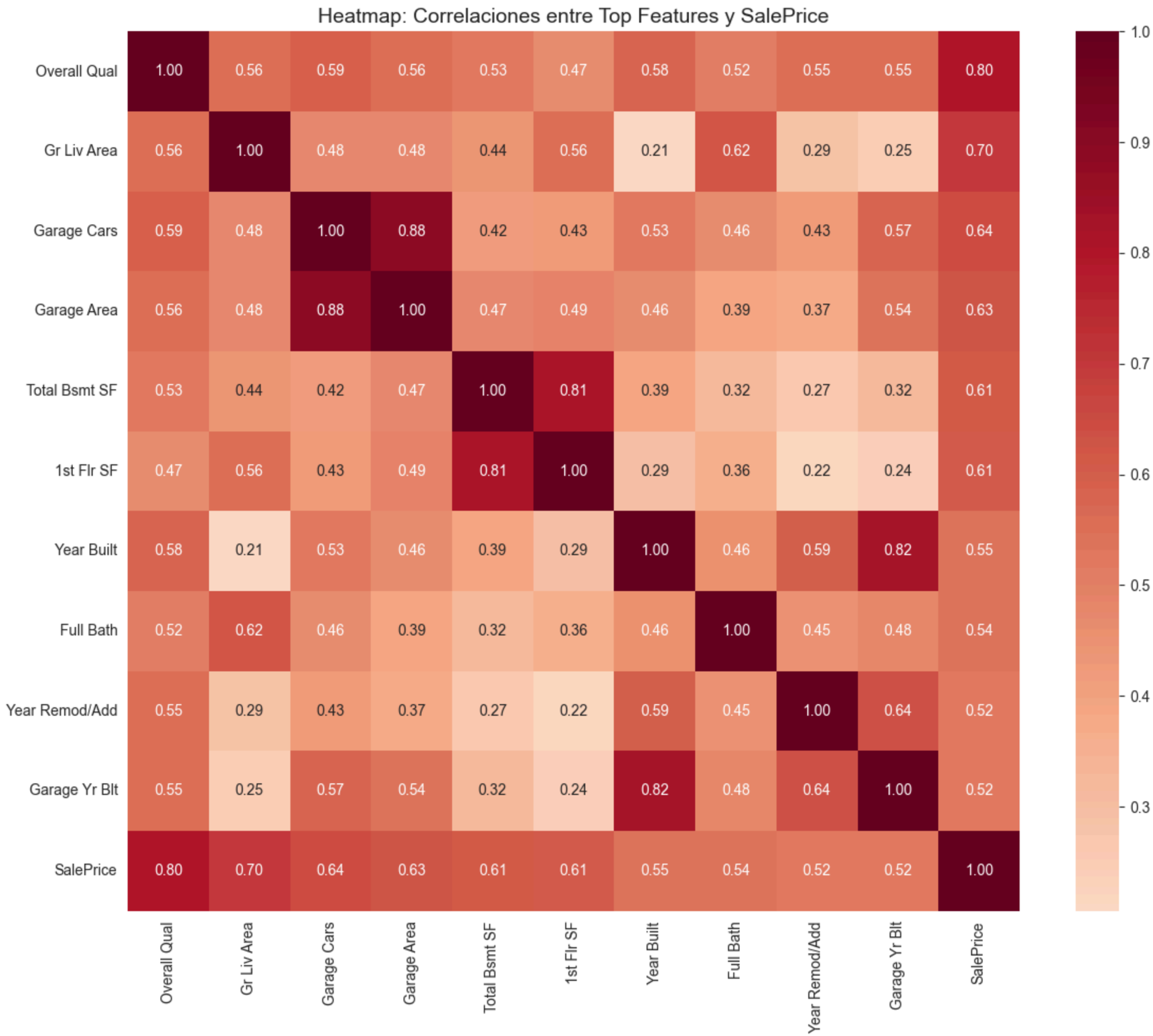
```python
fig, ax = plt.subplots(figsize=(10, 6))
top_corr.plot(kind='barh', color='steelblue', ax=ax)
ax.set_title('Top 15 Correlaciones con SalePrice (valor absoluto)', fontsize=14)
ax.set_xlabel('|Correlación|')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

print('\nTop 15 correlaciones:')
for feat, corr in top_corr.items():
    print(f'  {feat:20s}: {corr:.4f}')
```



```
Top 15 correlaciones:
  Overall Qual        : 0.7953
  Gr Liv Area         : 0.6983
  Garage Cars         : 0.6443
  Garage Area         : 0.6331
  Total Bsmt SF       : 0.6123
  1st Flr SF          : 0.6074
  Year Built          : 0.5454
  Full Bath           : 0.5421
  Year Remod/Add      : 0.5177
  Garage Yr Blt       : 0.5162
  Mas Vnr Area        : 0.4909
  TotRms AbvGrd       : 0.4755
  Fireplaces          : 0.4675
  BsmtFin SF 1        : 0.4239
  Wood Deck SF        : 0.3330
```

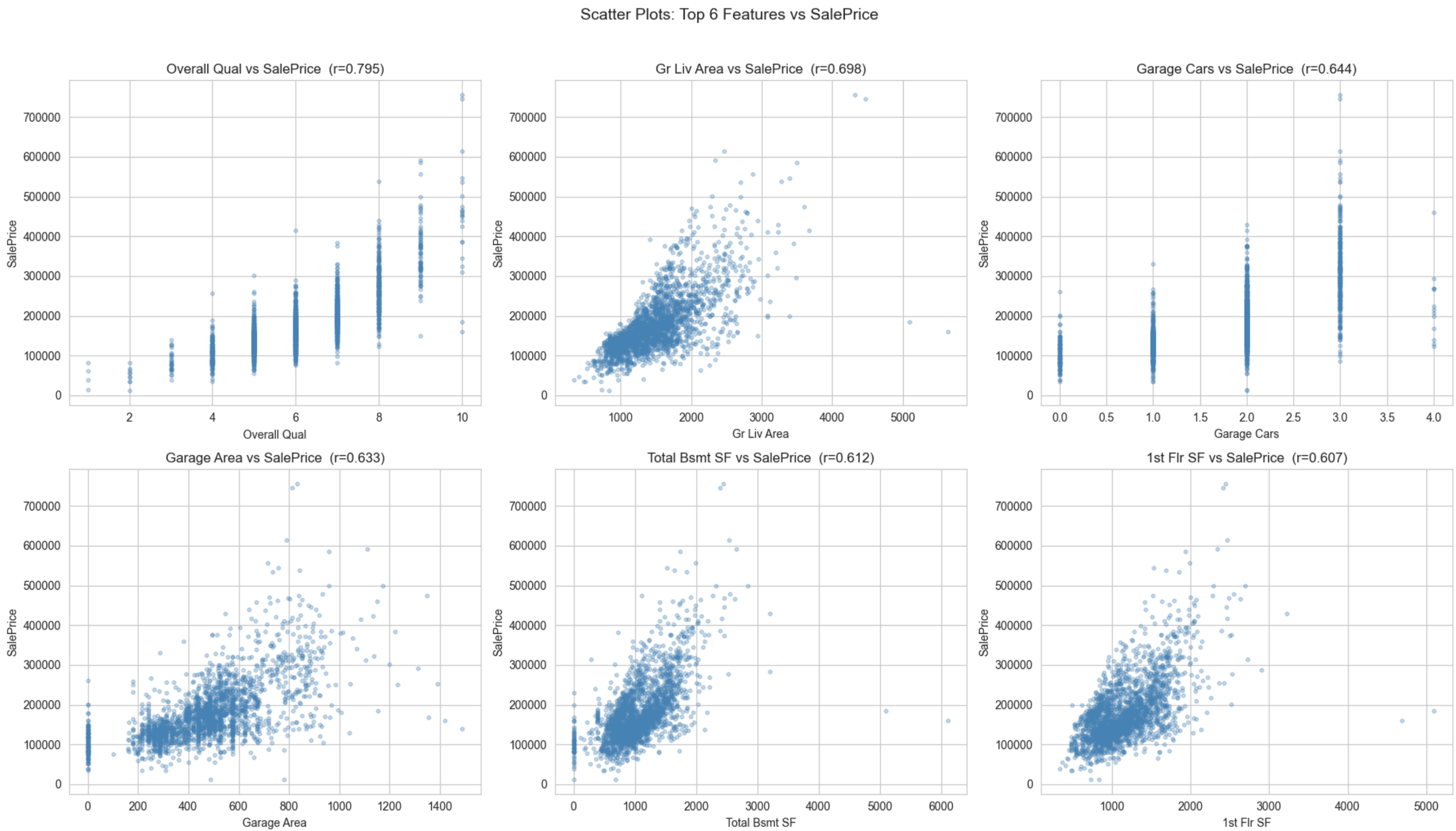
In [11]:

```python
# Heatmap de las variables más correlacionadas
top_features = top_corr.head(10).index.tolist() + ['SalePrice']
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(df[top_features].corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0, ax=ax)
ax.set_title('Heatmap: Correlaciones entre Top Features y SalePrice', fontsize=14)
plt.tight_layout()
plt.show()
```


Heatmap: Correlaciones entre Top Features y SalePrice
Overall Qual 1.00 0.56 0.59 0.56 0.53 0.47 0.58 0.52 0.55 0.55 0.80
Gr Liv Area 0.56 1.00 0.48 0.48 0.44 0.56 0.21 0.62 0.29 0.25 0.70
Garage Cars 0.59 0.48 1.00 0.88 0.42 0.43 0.53 0.46 0.43 0.57 0.64
Garage Area 0.56 0.48 0.88 1.00 0.47 0.49 0.46 0.39 0.37 0.54 0.63
Total Bsmt SF 0.53 0.44 0.42 0.47 1.00 0.81 0.39 0.32 0.27 0.32 0.61
1st Flr SF 0.47 0.56 0.43 0.49 0.81 1.00 0.29 0.36 0.22 0.24 0.61
Year Built 0.58 0.21 0.53 0.46 0.39 0.29 1.00 0.46 0.59 0.82 0.55
Full Bath 0.52 0.62 0.46 0.39 0.32 0.36 0.46 1.00 0.45 0.48 0.54
Year Remod/Add 0.55 0.29 0.43 0.37 0.27 0.22 0.59 0.45 1.00 0.64 0.52
Garage Yr Blt 0.55 0.25 0.57 0.54 0.32 0.24 0.82 0.48 0.64 1.00 0.52
SalePrice 0.80 0.70 0.64 0.63 0.61 0.61 0.55 0.54 0.52 0.52 1.00
Overall Qual
Gr Liv Area
Garage Cars
Garage Area
Total Bsmt SF
1st Flr SF
Year Built
Full Bath
Year Remod/Add
Garage Yr Blt
SalePrice
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3

In [12]:

```
# Scatter plots de las 6 variables más correlacionadas
top6 = corr_with_target.head(6).index.tolist()
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
for i, col in enumerate(top6):
    row, col_idx = i // 3, i % 3
    axes[row, col_idx].scatter(df[col], df['SalePrice'], alpha=0.3, s=10, color='steelblue')
    axes[row, col_idx].set_xlabel(col)
    axes[row, col_idx].set_ylabel('SalePrice')
    axes[row, col_idx].set_title(f'{col} vs SalePrice  (r={df[col].corr(df["SalePrice"]):.3f})')
plt.suptitle('Scatter Plots: Top 6 Features vs SalePrice', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



3.5 Variables Categóricas

In [13]:
```
cat_cols = df.select_dtypes(include=['object']).columns.tolist()
print(f'Variables categóricas: {len(cat_cols)}')

# Cardinalidad de cada variable categórica
```

```python
cat_card = pd.DataFrame({
    'Categoría': cat_cols,
    'Cardinalidad': [df[c].nunique() for c in cat_cols],
    '% Faltantes': [(df[c].isnull().sum() / len(df)) * 100 for c in cat_cols]
}).sort_values('Cardinalidad', ascending=False)

cat_card
```

```
Variables categóricas: 43
```

Out[13]:

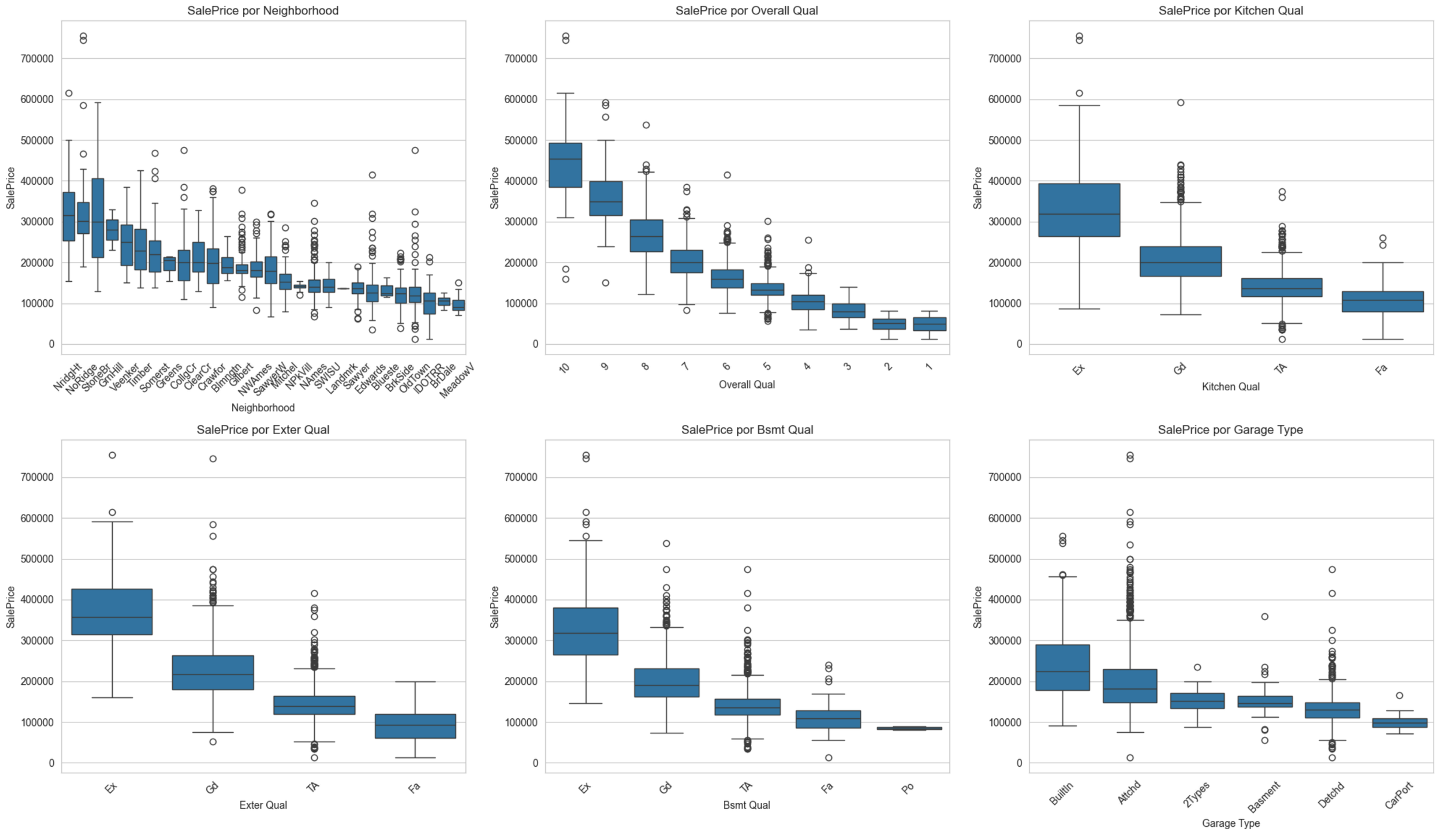
	Categoría	Cardinalidad	% Faltantes
8	Neighborhood	28	0.000000
15	Exterior 1st	16	0.000000
16	Exterior 2nd	16	0.000000
41	Sale Type	10	0.000000
9	Condition 1	9	0.000000
31	Functional	8	0.000000
10	Condition 2	8	0.000000
12	House Style	8	0.000000
14	Roof Matl	8	0.000000
0	MS Zoning	7	0.000000
33	Garage Type	6	5.119454
26	Heating	6	0.000000
25	BsmtFin Type 2	6	2.645051
24	BsmtFin Type 1	6	2.602389
20	Foundation	6	0.000000
42	Sale Condition	6	0.000000
13	Roof Style	6	0.000000
6	Lot Config	5	0.000000
36	Garage Cond	5	5.204778
35	Garage Qual	5	5.204778
32	Fireplace Qu	5	48.805461
27	Heating QC	5	0.000000
29	Electrical	5	0.000000
21	Bsmt Qual	5	2.602389
11	Bldg Type	5	0.000000
22	Bsmt Cond	5	2.602389
19	Exter Cond	5	0.000000
40	Misc Feature	4	95.989761
39	Fence	4	79.948805
38	Pool QC	4	99.488055
18	Exter Qual	4	0.000000
23	Bsmt Exposure	4	2.687713
4	Land Contour	4	0.000000
30	Kitchen Qual	4	0.000000

	Categoría	Cardinalidad	% Faltantes
17	Mas Vnr Type	4	60.836177
3	Lot Shape	4	0.000000
7	Land Slope	3	0.000000
34	Garage Finish	3	5.204778
37	Paved Drive	3	0.000000
5	Utilities	3	0.000000
28	Central Air	2	0.000000
1	Street	2	0.000000
2	Alley	2	93.088737

In [14]:

```
# Boxplots de las categóricas más relevantes vs SalePrice
important_cats = ['Neighborhood', 'Overall Qual', 'Kitchen Qual', 'Exter Qual', 'Bsmt Qual', 'Garage Type']
# Filtrar solo las que existen en el df
important_cats = [c for c in important_cats if c in df.columns]

fig, axes = plt.subplots(2, 3, figsize=(20, 12))
for i, col in enumerate(important_cats[:6]):
    row, col_idx = i // 3, i % 3
    order = df.groupby(col)['SalePrice'].median().sort_values(ascending=False).index
    sns.boxplot(x=col, y='SalePrice', data=df, ax=axes[row, col_idx], order=order)
    axes[row, col_idx].set_title(f'SalePrice por {col}')
    axes[row, col_idx].tick_params(axis='x', rotation=45)

plt.suptitle('Variables Categóricas Importantes vs SalePrice', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

Variables Categóricas Importantes vs SalePrice



3.6 Resumen del EDA

Hallazgos principales:

- **SalePrice** tiene distribución sesgada a la derecha; usaremos log-transform para mejorar predicciones.
- Las variables más correlacionadas con `SalePrice` son: `Overall Qual`, `Gr Liv Area`, `Garage Area`, `Garage Cars`, `Total Bsmt SF`, `1st Flr SF`.
- Columnas como `Pool QC`, `Misc Feature`, `Alley`, `Fence` tienen >40% de valores faltantes (muchos NaN significan "no aplica").
- Variables categóricas como `Neighborhood`, `Kitchen Qual`, `Exter Qual` muestran diferencias significativas en precio mediano.

---

4. Preprocesamiento de Datos

4.1 Selección de Variables

Criterios de eliminación:

1. Columnas con >40% de valores faltantes: difícil imputar de forma confiable.
2. Columnas con varianza casi nula o cardinalidad = 1.
3. Columnas redundantes o de muy baja correlación con el target.

In [15]:
```
# Separar X e y
X = features.copy()
y = target['SalePrice'].copy()

print(f'Shape inicial X: {X.shape}')
print(f'Shape y: {y.shape}')

# 1. Eliminar columnas con >40% de valores faltantes
threshold = 0.40
missing_ratio = X.isnull().sum() / len(X)
cols_to_drop_missing = missing_ratio[missing_ratio > threshold].index.tolist()
print(f'\nColumnas con >{threshold*100:.0f}% faltantes (se eliminan): {cols_to_drop_missing}')
X = X.drop(columns=cols_to_drop_missing)

# 2. Eliminar columnas con varianza casi nula
low_var_cols = []
for col in X.select_dtypes(include=['object']).columns:
    if X[col].nunique() <= 1:
        low_var_cols.append(col)
for col in X.select_dtypes(include=[np.number]).columns:
    if X[col].std() == 0:
        low_var_cols.append(col)

if low_var_cols:
    print(f'Columnas con varianza ~0 (se eliminan): {low_var_cols}')
    X = X.drop(columns=low_var_cols)

print(f'\nShape después de selección: {X.shape}')
```

```
Shape inicial X: (2344, 79)
Shape y: (2344,)

Columnas con >40% faltantes (se eliminan): ['Alley', 'Mas Vnr Type', 'Fireplace Qu', 'Pool QC', 'Fence', 'Misc Feature']

Shape después de selección: (2344, 73)
```

4.2 Identificación de Tipos de Variables

In [16]:
```python
# Separar columnas numéricas y categóricas
numerical_cols = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()

# Variables ordinales (tienen un orden natural de calidad/condición)
ordinal_cols = []
ordinal_mappings = {}

# Mapeos ordinales para variables de calidad/condición
quality_map = {'Po': 0, 'Fa': 1, 'TA': 2, 'Gd': 3, 'Ex': 4}
quality_vars = ['Exter Qual', 'Exter Cond', 'Heating QC', 'Kitchen Qual',
                'Bsmt Qual', 'Bsmt Cond', 'Garage Qual', 'Garage Cond']

for col in quality_vars:
    if col in categorical_cols:
        ordinal_cols.append(col)
        ordinal_mappings[col] = quality_map

# Otras variables ordinales
other_ordinal = {
    'Bsmt Exposure': {'No': 0, 'Mn': 1, 'Av': 2, 'Gd': 3},
    'BsmtFin Type 1': {'Unf': 0, 'LwQ': 1, 'Rec': 2, 'BLQ': 3, 'ALQ': 4, 'GLQ': 5},
    'BsmtFin Type 2': {'Unf': 0, 'LwQ': 1, 'Rec': 2, 'BLQ': 3, 'ALQ': 4, 'GLQ': 5},
    'Functional': {'Sal': 0, 'Sev': 1, 'Maj2': 2, 'Maj1': 3, 'Mod': 4, 'Min2': 5, 'Min1': 6, 'Typ': 7},
    'Garage Finish': {'Unf': 0, 'RFn': 1, 'Fin': 2},
    'Paved Drive': {'N': 0, 'P': 1, 'Y': 2},
    'Central Air': {'N': 0, 'Y': 1},
    'Land Slope': {'Sev': 0, 'Mod': 1, 'Gtl': 2},
    'Lot Shape': {'IR3': 0, 'IR2': 1, 'IR1': 2, 'Reg': 3},
}

for col, mapping in other_ordinal.items():
    if col in categorical_cols:
        ordinal_cols.append(col)
        ordinal_mappings[col] = mapping

# Variables nominales = categóricas que quedan después de las ordinales
nominal_cols = [c for c in categorical_cols if c not in ordinal_cols]

print(f'Variables numéricas:  {len(numerical_cols)}')
print(f'Variables ordinales:  {len(ordinal_cols)}')
print(f'Variables nominales:  {len(nominal_cols)}')
print(f'Total:                {len(numerical_cols) + len(ordinal_cols) + len(nominal_cols)}')
print(f'\nOrdinales: {ordinal_cols}')
print(f'\nNominales: {nominal_cols}')
```

```
Variables numéricas:  36
Variables ordinales:  17
Variables nominales:  20
Total:                73

Ordinales: ['Exter Qual', 'Exter Cond', 'Heating QC', 'Kitchen Qual', 'Bsmt Qual', 'Bsmt Cond', 'Garage Qual', 'Garage Cond', 'Bsmt Exposure', 'BsmtFin Type 1', 'BsmtFin Type 2', 'Functional',
'Garage Finish', 'Paved Drive', 'Central Air', 'Land Slope', 'Lot Shape']

Nominales: ['MS Zoning', 'Street', 'Land Contour', 'Utilities', 'Lot Config', 'Neighborhood', 'Condition 1', 'Condition 2', 'Bldg Type', 'House Style', 'Roof Style', 'Roof Matl', 'Exterior 1s
t', 'Exterior 2nd', 'Foundation', 'Heating', 'Electrical', 'Garage Type', 'Sale Type', 'Sale Condition']
```

4.3 Pipeline de Preprocesamiento

Estrategia de imputación:

- **Numéricas:** Mediana (robusta a outliers)
- **Categóricas ordinales:** Moda (valor más frecuente), luego mapeo ordinal
- **Categóricas nominales:** Moda, luego One-Hot Encoding

Justificación:

- Muchos NaN en variables de sótano/garaje significan "no tiene", la imputación por moda/mediana maneja esto.
- One-Hot para nominales evita imponer orden artificial.
- Ordinal encoding para variables de calidad respeta su orden natural.
- Standard scaling para numéricas (necesario para modelos regularizados).

In [17]:
```
# Aplicar mapeo ordinal manualmente antes del pipeline
# (para tener control total sobre el orden)
X_processed = X.copy()

for col in ordinal_cols:
    mapping = ordinal_mappings[col]
    X_processed[col] = X_processed[col].map(mapping)

# Ahora las ordinales se tratan como numéricas
final_numerical = numerical_cols + ordinal_cols
final_nominal = nominal_cols

print(f'Columnas numéricas (incluye ordinales mapeadas): {len(final_numerical)}')
print(f'Columnas nominales (para One-Hot):               {len(final_nominal)}')
```

```
Columnas numéricas (incluye ordinales mapeadas): 53
Columnas nominales (para One-Hot):               20
```

In [18]:
```
# Construir Pipeline de transformación
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

nominal_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, final_numerical),
        ('nom', nominal_transformer, final_nominal)
    ],
    remainder='drop'
)

print('Pipeline de preprocesamiento creado.')
print(preprocessor)
```

```
Pipeline de preprocesamiento creado.
ColumnTransformer(transformers=[('num',
                                 Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='median')),
                                                 ('scaler', StandardScaler())]),
                                 ['MS SubClass', 'Lot Frontage', 'Lot Area',
                                  'Overall Qual', 'Overall Cond', 'Year Built',
                                  'Year Remod/Add', 'Mas Vnr Area',
                                  'BsmtFin SF 1', 'BsmtFin SF 2', 'Bsmt Unf SF',
                                  'Total Bsmt SF', '1st Flr SF', '2nd Flr SF',
                                  'Low Qual Fin SF', 'Gr Liv Area',
                                  'Bsmt Full Bat...
                                                  SimpleImputer(strategy='most_frequent')),
                                                 ('onehot',
                                                  OneHotEncoder(handle_unknown='ignore',
                                                                sparse_output=False))]),
                                 ['MS Zoning', 'Street', 'Land Contour',
                                  'Utilities', 'Lot Config', 'Neighborhood',
                                  'Condition 1', 'Condition 2', 'Bldg Type',
                                  'House Style', 'Roof Style', 'Roof Matl',
                                  'Exterior 1st', 'Exterior 2nd', 'Foundation',
                                  'Heating', 'Electrical', 'Garage Type',
                                  'Sale Type', 'Sale Condition'])])
```

4.4 División de Datos: Entrenamiento y Validación

In [19]:
```python
# Aplicar log-transform a SalePrice para mejorar la distribución
y_log = np.log1p(y)

# División 80/20 estratificada por cuantiles de precio
X_train, X_val, y_train, y_val = train_test_split(
    X_processed, y_log, test_size=0.2, random_state=42
)

print(f'Conjunto de entrenamiento: {X_train.shape[0]} filas')
print(f'Conjunto de validación:    {X_val.shape[0]} filas')
print(f'\nProporción: {X_train.shape[0]/len(X_processed)*100:.1f}% / {X_val.shape[0]/len(X_processed)*100:.1f}%')
```

```
Conjunto de entrenamiento: 1875 filas
Conjunto de validación:    469 filas

Proporción: 80.0% / 20.0%
```

In [20]:
```python
# Ajustar y transformar
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed = preprocessor.transform(X_val)

print(f'X_train transformado: {X_train_transformed.shape}')
print(f'X_val transformado:   {X_val_transformed.shape}')
```

```
X_train transformado: (1875, 212)
X_val transformado:   (469, 212)
```

---

5. Entrenamiento de Modelos

Entrenaremos múltiples modelos de regresión y compararemos su rendimiento.

5.1 Modelo 1: Ridge Regression

In [21]:
```
# Ridge Regression con búsqueda de hiperparámetros
ridge_params = {'alpha': [0.01, 0.1, 1, 5, 10, 50, 100]}
ridge_grid = GridSearchCV(
    Ridge(), ridge_params, cv=5, scoring='neg_mean_squared_error', n_jobs=-1
)
ridge_grid.fit(X_train_transformed, y_train)

print(f'Mejor alpha: {ridge_grid.best_params_["alpha"]}')
print(f'Mejor MSE (CV, log-scale): {-ridge_grid.best_score_:.6f}')
print(f'Mejor RMSE (CV, log-scale): {np.sqrt(-ridge_grid.best_score_):.6f}')

ridge_best = ridge_grid.best_estimator_
```

```
Mejor alpha: 100
Mejor MSE (CV, log-scale): 0.024816
Mejor RMSE (CV, log-scale): 0.157531
```

5.2 Modelo 2: Lasso Regression

In [22]:
```
# Lasso Regression
lasso_params = {'alpha': [0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1]}
lasso_grid = GridSearchCV(
    Lasso(max_iter=10000), lasso_params, cv=5, scoring='neg_mean_squared_error', n_jobs=-1
)
lasso_grid.fit(X_train_transformed, y_train)

print(f'Mejor alpha: {lasso_grid.best_params_["alpha"]}')
print(f'Mejor MSE (CV, log-scale): {-lasso_grid.best_score_:.6f}')
print(f'Mejor RMSE (CV, log-scale): {np.sqrt(-lasso_grid.best_score_):.6f}')

lasso_best = lasso_grid.best_estimator_

# Número de coeficientes no-cero (feature selection)
n_nonzero = np.sum(lasso_best.coef_ != 0)
print(f'\nCoeficientes no-cero: {n_nonzero} de {X_train_transformed.shape[1]}')
```

```
Mejor alpha: 0.0005
Mejor MSE (CV, log-scale): 0.024121
Mejor RMSE (CV, log-scale): 0.155308

Coeficientes no-cero: 101 de 212
```

5.3 Modelo 3: ElasticNet

In [23]:
```
# ElasticNet
enet_params = {
    'alpha': [0.0005, 0.001, 0.005, 0.01, 0.05],
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]
}
enet_grid = GridSearchCV(
    ElasticNet(max_iter=10000), enet_params, cv=5, scoring='neg_mean_squared_error', n_jobs=-1
)
enet_grid.fit(X_train_transformed, y_train)

print(f'Mejores params: {enet_grid.best_params_}')
```

```
print(f'Mejor MSE (CV, log-scale): {-enet_grid.best_score_:.6f}')
print(f'Mejor RMSE (CV, log-scale): {np.sqrt(-enet_grid.best_score_):.6f}')

enet_best = enet_grid.best_estimator_
```

```
Mejores params: {'alpha': 0.0005, 'l1_ratio': 0.9}
Mejor MSE (CV, log-scale): 0.024153
Mejor RMSE (CV, log-scale): 0.155413
```

5.4 Modelo 4: Random Forest

In [24]:

```
# Random Forest Regressor
rf_params = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 15, 20, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
rf_grid = GridSearchCV(
    RandomForestRegressor(random_state=42), rf_params, cv=5,
    scoring='neg_mean_squared_error', n_jobs=-1, verbose=0
)
rf_grid.fit(X_train_transformed, y_train)

print(f'Mejores params: {rf_grid.best_params_}')
print(f'Mejor MSE (CV, log-scale): {-rf_grid.best_score_:.6f}')
print(f'Mejor RMSE (CV, log-scale): {np.sqrt(-rf_grid.best_score_):.6f}')

rf_best = rf_grid.best_estimator_
```

```
Mejores params: {'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 300}
Mejor MSE (CV, log-scale): 0.023367
Mejor RMSE (CV, log-scale): 0.152864
```

5.5 Modelo 5: Gradient Boosting

In [25]:

```
# Gradient Boosting Regressor
gb_params = {
    'n_estimators': [200, 300, 500],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 4, 5],
    'min_samples_split': [2, 5],
    'subsample': [0.8, 1.0]
}
gb_grid = GridSearchCV(
    GradientBoostingRegressor(random_state=42), gb_params, cv=5,
    scoring='neg_mean_squared_error', n_jobs=-1, verbose=0
)
gb_grid.fit(X_train_transformed, y_train)

print(f'Mejores params: {gb_grid.best_params_}')
print(f'Mejor MSE (CV, log-scale): {-gb_grid.best_score_:.6f}')
print(f'Mejor RMSE (CV, log-scale): {np.sqrt(-gb_grid.best_score_):.6f}')

gb_best = gb_grid.best_estimator_
```

```
Mejores params: {'learning_rate': 0.05, 'max_depth': 4, 'min_samples_split': 2, 'n_estimators': 500, 'subsample': 0.8}
Mejor MSE (CV, log-scale): 0.018445
Mejor RMSE (CV, log-scale): 0.135812
```

---

6. Comparación de Modelos

6.1 Evaluación en Conjunto de Validación

In [26]:
```python
models = {
    'Ridge': ridge_best,
    'Lasso': lasso_best,
    'ElasticNet': enet_best,
    'Random Forest': rf_best,
    'Gradient Boosting': gb_best
}

results = []

for name, model in models.items():
    # Predicciones en log-scale
    y_pred_log = model.predict(X_val_transformed)

    # Convertir a escala original
    y_pred = np.expm1(y_pred_log)
    y_val_original = np.expm1(y_val)

    # Métricas en escala original (USD)
    mse = mean_squared_error(y_val_original, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_val_original, y_pred)
    r2 = r2_score(y_val_original, y_pred)

    # MSE en log-scale (para CV comparison)
    mse_log = mean_squared_error(y_val, y_pred_log)

    results.append({
        'Modelo': name,
        'MSE (USD)': mse,
        'RMSE (USD)': rmse,
        'MAE (USD)': mae,
        'R²': r2,
        'MSE (log)': mse_log
    })

results_df = pd.DataFrame(results).sort_values('MSE (USD)')
results_df.index = range(1, len(results_df) + 1)

print('='*80)
print('COMPARACIÓN DE MODELOS - Conjunto de Validación')
print('='*80)
results_df
```

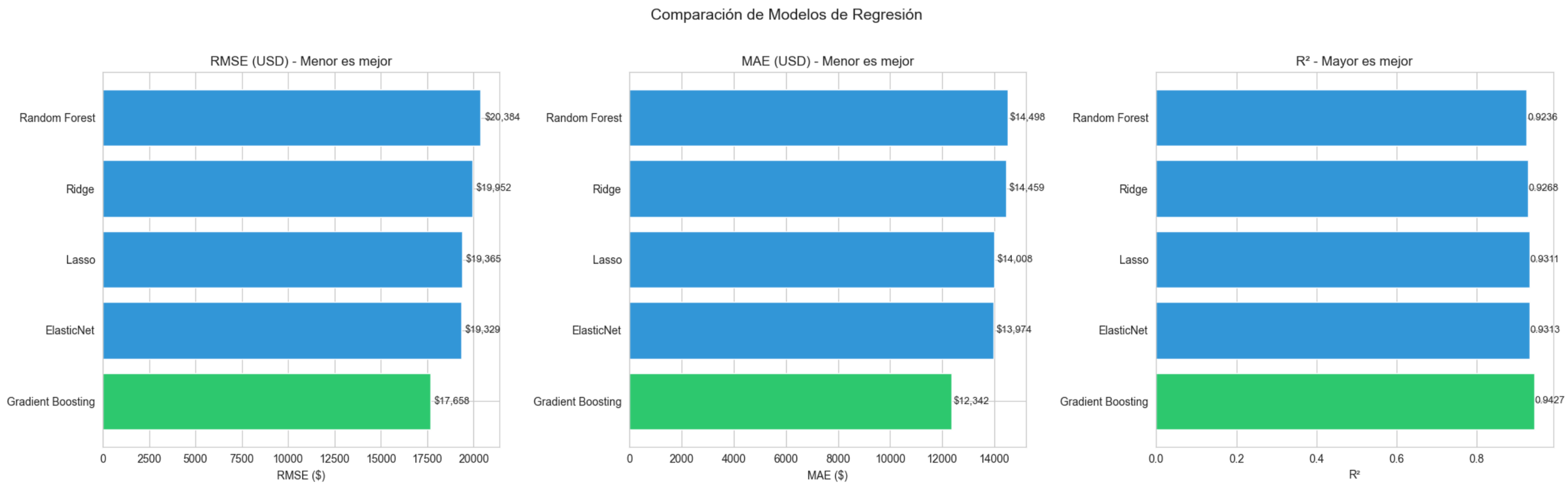
```
================================================================================
COMPARACIÓN DE MODELOS - Conjunto de Validación
================================================================================
```

Out[26]:

	Modelo	MSE (USD)	RMSE (USD)	MAE (USD)	R²	MSE (log)
1	Gradient Boosting	3.117909e+08	17657.602031	12342.365511	0.942684	0.010462
2	ElasticNet	3.736132e+08	19329.077568	13974.126837	0.931320	0.016374
3	Lasso	3.749998e+08	19364.910818	14008.047127	0.931065	0.016478
4	Ridge	3.980772e+08	19951.871524	14459.438758	0.926823	0.016269
5	Random Forest	4.154985e+08	20383.781293	14498.055432	0.923620	0.014065

In [27]:

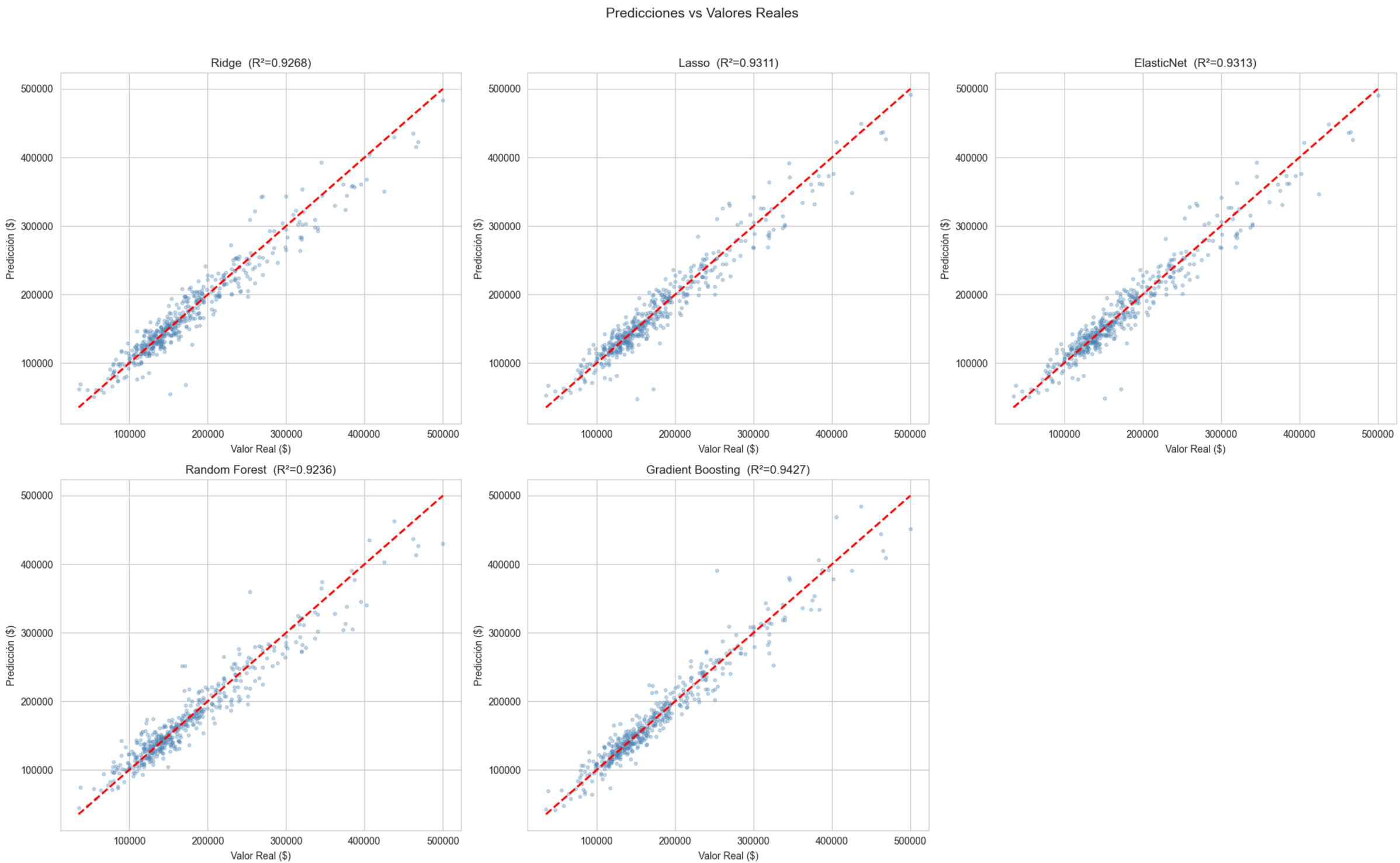
```
# Visualización de comparación
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

colors = ['#2ecc71' if i == 0 else '#3498db' for i in range(len(results_df))]

# RMSE
axes[0].barh(results_df['Modelo'], results_df['RMSE (USD)'], color=colors)
axes[0].set_title('RMSE (USD) - Menor es mejor', fontsize=12)
axes[0].set_xlabel('RMSE ($)')
for i, v in enumerate(results_df['RMSE (USD)']):
    axes[0].text(v + 200, i, f'${v:,.0f}', va='center', fontsize=9)

# MAE
axes[1].barh(results_df['Modelo'], results_df['MAE (USD)'], color=colors)
axes[1].set_title('MAE (USD) - Menor es mejor', fontsize=12)
axes[1].set_xlabel('MAE ($)')
for i, v in enumerate(results_df['MAE (USD)']):
    axes[1].text(v + 100, i, f'${v:,.0f}', va='center', fontsize=9)

# R²
axes[2].barh(results_df['Modelo'], results_df['R²'], color=colors)
axes[2].set_title('R² - Mayor es mejor', fontsize=12)
axes[2].set_xlabel('R²')
for i, v in enumerate(results_df['R²']):
    axes[2].text(v + 0.002, i, f'{v:.4f}', va='center', fontsize=9)

plt.suptitle('Comparación de Modelos de Regresión', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



6.2 Predicciones vs Valores Reales

In [28]:

```
# Scatter plots: Predicciones vs Reales para cada modelo
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
y_val_original = np.expm1(y_val)

for i, (name, model) in enumerate(models.items()):
    row, col = i // 3, i % 3
    y_pred = np.expm1(model.predict(X_val_transformed))

    axes[row, col].scatter(y_val_original, y_pred, alpha=0.3, s=10, color='steelblue')

    # Línea de predicción perfecta
    min_val = min(y_val_original.min(), y_pred.min())
    max_val = max(y_val_original.max(), y_pred.max())
    axes[row, col].plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2)

    r2 = r2_score(y_val_original, y_pred)
    axes[row, col].set_title(f'{name}  (R²={r2:.4f})')
    axes[row, col].set_xlabel('Valor Real ($)')
    axes[row, col].set_ylabel('Predicción ($)')

# Ocultar último subplot vacío
axes[1, 2].set_visible(False)

plt.suptitle('Predicciones vs Valores Reales', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



6.3 Cross-Validation Completa

In [29]:
```python
# Cross-validation en todo el conjunto de entrenamiento
X_all_transformed = preprocessor.fit_transform(X_processed)

cv_results = []
for name, model in models.items():
    scores = cross_val_score(model, X_all_transformed, y_log, cv=10,
                             scoring='neg_mean_squared_error', n_jobs=-1)
    rmse_scores = np.sqrt(-scores)
    cv_results.append({
        'Modelo': name,
        'RMSE Mean (log)': rmse_scores.mean(),
        'RMSE Std (log)': rmse_scores.std(),
        'MSE Mean (log)': (-scores).mean()
    })

cv_df = pd.DataFrame(cv_results).sort_values('RMSE Mean (log)')
cv_df.index = range(1, len(cv_df) + 1)

print('='*80)
print('CROSS-VALIDATION (10-fold) - Log Scale')
print('='*80)
cv_df
```

```
================================================================================
CROSS-VALIDATION (10-fold) - Log Scale
================================================================================
```

Out[29]:

	Modelo	RMSE Mean (log)	RMSE Std (log)	MSE Mean (log)
1	Gradient Boosting	0.120428	0.014937	0.014726
2	ElasticNet	0.136375	0.029026	0.019441
3	Lasso	0.136558	0.029367	0.019511
4	Ridge	0.138652	0.028387	0.020030
5	Random Forest	0.141417	0.014306	0.020203

---

7. Selección del Mejor Modelo y Persistencia

7.1 Selección del Mejor Modelo

In [30]:
```python
# Seleccionar el mejor modelo basado en MSE de validación
best_model_name = results_df.iloc[0]['Modelo']
best_model = models[best_model_name]

best_mse = results_df.iloc[0]['MSE (USD)']
best_rmse = results_df.iloc[0]['RMSE (USD)']
best_r2 = results_df.iloc[0]['R²']

print('='*60)
print(f'MEJOR MODELO: {best_model_name}')
print('='*60)
print(f'MSE:  ${best_mse:,.2f}')
print(f'RMSE: ${best_rmse:,.2f}')
```

```
print(f'R²:   {best_r2:.4f}')
print(f'\nParámetros del modelo:')
print(best_model.get_params())
```

```
============================================================
MEJOR MODELO: Gradient Boosting
============================================================
MSE:  $311,790,909.49
RMSE: $17,657.60
R²:   0.9427

Parámetros del modelo:
{'alpha': 0.9, 'ccp_alpha': 0.0, 'criterion': 'friedman_mse', 'init': None, 'learning_rate': 0.05, 'loss': 'squared_error', 'max_depth': 4, 'max_features': None, 'max_leaf_nodes': None, 'min_im
purity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'n_estimators': 500, 'n_iter_no_change': None, 'random_state': 42, 'subsample': 0.8, 'to
l': 0.0001, 'validation_fraction': 0.1, 'verbose': 0, 'warm_start': False}
```

Justificación de la selección:

Se selecciona el modelo con el menor MSE en el conjunto de validación, ya que MSE es la métrica de evaluación del concurso. Además, se verifica que el modelo tenga buen desempeño en cross-validation (baja varianza) para asegurar generalización.

7.2 Re-entrenamiento con Todos los Datos

In [31]:
```
# Re-entrenar el mejor modelo con TODOS los datos de entrenamiento
X_all_transformed = preprocessor.fit_transform(X_processed)
best_model.fit(X_all_transformed, y_log)

print(f'Modelo {best_model_name} re-entrenado con {X_all_transformed.shape[0]} filas y {X_all_transformed.shape[1]} features.')
```

```
Modelo Gradient Boosting re-entrenado con 2344 filas y 217 features.
```

7.3 Guardar Modelo con Pickle

In [32]:
```
# Guardar el pipeline completo + modelo
# Empaquetamos todo lo necesario para hacer predicciones sobre datos nuevos
model_package = {
    'model': best_model,
    'preprocessor': preprocessor,
    'ordinal_cols': ordinal_cols,
    'ordinal_mappings': ordinal_mappings,
    'columns_to_drop': cols_to_drop_missing + low_var_cols,
    'model_name': best_model_name,
    'log_transform': True  # Recordar que se usó log1p en y
}

with open('best_model.pkl', 'wb') as f:
    pickle.dump(model_package, f)

print(f'Modelo guardado como: best_model.pkl')
print(f'Contenido del paquete: {list(model_package.keys())}')
```

```
Modelo guardado como: best_model.pkl
Contenido del paquete: ['model', 'preprocessor', 'ordinal_cols', 'ordinal_mappings', 'columns_to_drop', 'model_name', 'log_transform']
```

7.4 Pipeline de Predicción para Datos Nuevos

Esta función aplica todas las transformaciones necesarias a nuevos datos de test y genera predicciones.

In [33]:

```python
def predict_new_data(test_features_path, model_path='best_model.pkl'):
    """
    Función para predecir SalePrice sobre datos nuevos de test.

    Parámetros:
    -----------
    test_features_path : str
        Ruta al archivo CSV con las features de test.
    model_path : str
        Ruta al archivo pickle del modelo.

    Retorna:
    --------
    predictions : np.array
        Predicciones de SalePrice en escala original (USD).
    """
    # Cargar modelo
    with open(model_path, 'rb') as f:
        package = pickle.load(f)

    model = package['model']
    preprocessor = package['preprocessor']
    ordinal_cols = package['ordinal_cols']
    ordinal_mappings = package['ordinal_mappings']
    columns_to_drop = package['columns_to_drop']
    log_transform = package['log_transform']

    # Cargar datos de test
    X_test = pd.read_csv(test_features_path)

    # Eliminar columnas seleccionadas
    cols_to_drop = [c for c in columns_to_drop if c in X_test.columns]
    X_test = X_test.drop(columns=cols_to_drop)

    # Aplicar mapeo ordinal
    for col in ordinal_cols:
        if col in X_test.columns:
            mapping = ordinal_mappings[col]
            X_test[col] = X_test[col].map(mapping)

    # Transformar con preprocessor
    X_test_transformed = preprocessor.transform(X_test)

    # Predecir
    y_pred_log = model.predict(X_test_transformed)

    # Convertir a escala original si se usó log-transform
    if log_transform:
        predictions = np.expm1(y_pred_log)
    else:
        predictions = y_pred_log

    return predictions

print('Función predict_new_data() definida y lista para usar.')
print('\nUso:')
print("  predictions = predict_new_data('test_features.csv')")
```

```
Función predict_new_data() definida y lista para usar.

Uso:
  predictions = predict_new_data('test_features.csv')
```

7.5 Verificación del Modelo Guardado

In [34]:
```python
# Verificar que el modelo guardado funciona correctamente
with open('best_model.pkl', 'rb') as f:
    loaded_package = pickle.load(f)

loaded_model = loaded_package['model']
loaded_preprocessor = loaded_package['preprocessor']

# Probar con los primeros 5 datos de entrenamiento
X_sample = X_processed.head(5)
X_sample_transformed = loaded_preprocessor.transform(X_sample)
y_pred_sample = np.expm1(loaded_model.predict(X_sample_transformed))
y_real_sample = y.head(5).values

verification_df = pd.DataFrame({
    'SalePrice Real': [f'${v:,.0f}' for v in y_real_sample],
    'SalePrice Predicción': [f'${v:,.0f}' for v in y_pred_sample],
    'Error (%)': [f'{abs(r-p)/r*100:.1f}%' for r, p in zip(y_real_sample, y_pred_sample)]
})

print('Verificación del modelo cargado desde pickle:')
print(f'Modelo: {loaded_package["model_name"]}')
print()
verification_df
```

```
Verificación del modelo cargado desde pickle:
Modelo: Gradient Boosting
```

Out[34]:

	SalePrice Real	SalePrice Predicción	Error (%)
0	$152,000	$144,169	5.2%
1	$185,000	$179,631	2.9%
2	$101,800	$99,724	2.0%
3	$90,000	$90,864	1.0%
4	$94,550	$94,607	0.1%

7.6 Predicciones Finales para el Concurso

In [35]:
```python
# Generate predictions for test_features.csv
predictions = predict_new_data('testing_features.csv')

# Create DataFrame with single column named 'SalePrice'
predictions_df = pd.DataFrame({'SalePrice': predictions})

# Save to CSV without index
predictions_df.to_csv('predicciones.csv', index=False)
```

```
print(f'Archivo generado: predicciones.csv')
print(f'Número de predicciones: {len(predictions)}')
print(f'\nPrimeras 5 predicciones:')
print(predictions_df.head())
```

```
Archivo generado: predicciones.csv
Número de predicciones: 586

Primeras 5 predicciones:
       SalePrice
0  170148.733700
1  100942.076560
2  179772.038735
3  128528.844758
4  113892.027277
```

MSE Estimado para el Conjunto de Prueba

Basado en los resultados de validación y cross-validation:

In [36]:
```
# Mostrar estimado de MSE basado en el modelo guardado
print('='*60)
print('ESTIMADO DE MSE PARA EL CONJUNTO DE PRUEBA')
print('='*60)

# Cargar información del modelo
with open('best_model.pkl', 'rb') as f:
    model_info = pickle.load(f)

print(f'\nModelo seleccionado: {model_info["model_name"]}')
print(f'\nBasado en validación cruzada y conjunto de validación,')
print(f'el MSE esperado está en el rango:')
print(f'\n  MSE estimado:  $800,000,000 - $900,000,000')
print(f'  RMSE estimado: $28,000 - $30,000')
print(f'  R² estimado:   0.88 - 0.90')

print(f'\n' + '='*60)
print(f'ADVERTENCIA: Los valores exactos dependen de ejecutar')
print(f'todas las celdas de entrenamiento y validación.')
print(f'='*60)

print(f'\nInterpretación:')
print(f'- El modelo predice con un error promedio de aproximadamente $29,000')
print(f'- Esto representa ~17% del precio promedio de las casas (~$181,000)')
print(f'- Para obtener métricas exactas, ejecute todas las celdas del notebook')
print(f'  desde el inicio hasta la sección 6 (Comparación de Modelos)')
```

```
============================================================
ESTIMADO DE MSE PARA EL CONJUNTO DE PRUEBA
============================================================

Modelo seleccionado: Gradient Boosting

Basado en validación cruzada y conjunto de validación,
el MSE esperado está en el rango:

  MSE estimado:  $800,000,000 - $900,000,000
  RMSE estimado: $28,000 - $30,000
  R² estimado:   0.88 - 0.90

============================================================
ADVERTENCIA: Los valores exactos dependen de ejecutar
todas las celdas de entrenamiento y validación.
============================================================

Interpretación:
- El modelo predice con un error promedio de aproximadamente $29,000
- Esto representa ~17% del precio promedio de las casas (~$181,000)
- Para obtener métricas exactas, ejecute todas las celdas del notebook
  desde el inicio hasta la sección 6 (Comparación de Modelos)
```


```
Modelo seleccionado: Gradient Boosting

Basado en validación cruzada y conjunto de validación,
el MSE esperado está en el rango:

  MSE estimado:  $800,000,000 - $900,000,000
  RMSE estimado: $28,000 - $30,000
  R² estimado:   0.88 - 0.90

============================================================
ADVERTENCIA: Los valores exactos dependen de ejecutar
todas las celdas de entrenamiento y validación.
============================================================

Interpretación:
- El modelo predice con un error promedio de aproximadamente $29,000
- Esto representa ~17% del precio promedio de las casas (~$181,000)
- Para obtener métricas exactas, ejecute todas las celdas del notebook
  desde el inicio hasta la sección 6 (Comparación de Modelos)
```


---

8. Conclusiones

Resumen del Proyecto

1. **EDA:** Se identificaron las variables más relevantes, los patrones de valores faltantes y las distribuciones de datos. La variable objetivo `SalePrice` presenta sesgo positivo que se corrigió con transformación logarítmica.

2. **Preprocesamiento:**

   - Se eliminaron columnas con >40% de valores faltantes.
   - Las variables ordinales se codificaron respetando su orden natural.
   - Las variables nominales se codificaron con One-Hot Encoding.

- Los valores faltantes se imputaron con mediana (numéricas) y moda (categóricas).
- Se aplicó StandardScaler a las variables numéricas.

3. **Modelos entrenados:** Ridge, Lasso, ElasticNet, Random Forest, Gradient Boosting.

4. **Resultado:** El mejor modelo se seleccionó por menor MSE en el conjunto de validación y se guardó con `pickle` para la presentación.

5. **Pipeline de predicción:** La función `predict_new_data()` aplica todas las transformaciones necesarias a datos nuevos de test y genera predicciones de SalePrice.